

System-Level Implementation of a Real-Time Terminal Application using Custom Memory and I/O Management

Asmit Kumar, Vaishnav Verma

Department of Computer Science

I. ABSTRACT

This project presents the development of a real-time Snake game implemented in C, focusing on low-level system interactions. The application avoids standard high-level libraries where possible, instead utilizing custom-built modules for memory management, mathematical operations, and terminal I/O. The implementation includes three difficulty modes, a persistent scoring system, and optimized rendering techniques to ensure high performance in a terminal environment.

II. METHODOLOGY

The system architecture is divided into specialized modules. The following sections detail the implementation across five critical functional areas:

A. Memory Management

A custom first-fit heap allocator was implemented to manage a pre-allocated 64KB memory pool.

- Allocation:** `my_alloc` uses a linked-list of `BlockHeader` structures to track free and occupied segments.
- Fragmentation Control:** `my_dealloc` includes a coalescing mechanism that merges adjacent free blocks to maximize contiguous space.
- Efficiency:** This approach eliminates the overhead of frequent `sbrk` or `malloc` system calls during the game loop.

B. I/O Management

Direct terminal control was achieved using POSIX `termios` and ANSI escape sequences.

- Input:** The keyboard is configured for non-blocking, non-canonical input using `tcsetattr` and `fcntl(0, NONBLOCK)`, enabling real-time direction changes without the need for the Enter key.
- Rendering:** The application utilizes the Alternate Screen Buffer (`\033[?1049h`) to provide a clean, dedicated viewport. ANSI escape codes handle precise cursor positioning (`\033[H`) to eliminate screen flickering.

C. Process Management

The application follows a single-threaded state machine model.

- Timing:** A controlled loop utilizes `usleep` to regulate game ticks.
- Normalization:** Tick delays are dynamically adjusted (1.5x factor for vertical moves) to compensate for the character aspect ratio of terminal fonts, ensuring uniform snake speed in all directions.
- State Control:** Transitions between menus, active gameplay, and game-over states are managed via a centralized dispatcher in `main.c`.

D. File Systems

Persistence is handled via local file I/O for score tracking.

- **Storage:** High scores are stored in `scores.dat` using `fprintf` and retrieved using `fscanf`.
- **Management:** The system maintains a top-3 leaderboard per difficulty mode, utilizing a localized bubble sort to rank entries before committing to disk.

E. Security and Error Handling

The implementation prioritizes stability through defensive programming.

- **Boundary Safety:** All snake movements are validated against board dimensions using `my_inbounds`.
- **Memory Safety:** Every allocation request in `game.c` is checked for `NULL` returns to prevent crashes in out-of-memory conditions.
- **Robust Exit:** A restoration routine in `main.c` ensures that `tcsetattr` is called to return the terminal to its original state, even upon unexpected termination.

III. FUTURE SCOPE

Future enhancements could include the implementation of multi-threaded rendering to further decouple I/O from game logic, and the introduction of advanced path-finding algorithms for autonomous obstacles. Additionally, transitioning the score file to a binary format could improve data integrity.

IV. REFERENCES

- [1] IEEE Standard for Information Technology - POSIX, IEEE Std 1003.1.
- [2] B. W. Kernighan and D. M. Ritchie, "The C Programming Language," 2nd ed., 1988.
- [3] VT100 User Manual, Digital Equipment Corporation, 1978.
- [4] "Managing Memory in C," IEEE Software, Vol. 10, No. 2, 1993.